



Rising Waters

Intelligent Flood Prediction System

Project Documentation

Project Description

Rising Waters is a Machine Learning-powered flood prediction platform that analyzes historical meteorological and hydrological data to compute real-time flood risk assessments for administrative subdivisions across India. The platform's core Flood Risk Assessment module accepts measurements such as average temperature, humidity, cloud cover, and multiple cumulative rainfall figures (annual, seasonal, and monthly), and returns a clear risk classification, a confidence score, and the key rainfall metric driving the decision.

At the heart of the system is a Decision Tree Classifier, trained on more than a century of sub-divisional meteorological records, which achieves 96.55% accuracy on historical validation data. Built with Flask and served locally (with public exposure available via Ngrok), Rising Waters combines a fast prediction engine with an informative, multi-page web experience covering the model's purpose, features, development workflow, real-world use cases, and a contact channel for feedback and integrations.

The platform is designed for meteorologists, disaster-management authorities, and emergency-relief coordinators who need fast, data-backed early warnings — giving response teams the lead time to construct flood barriers, evacuate residents, and position rescue resources before water levels become critical.

Scenarios

Scenario 1: A meteorologist monitoring the monsoon season selects the Andaman & Nicobar Islands subdivision and loads the built-in "Flood Demo" preset. With a Jun-Sep cumulative rainfall of 2500 mm — far above the model's internal threshold — Rising Waters instantly returns a "Flood Risk Detected" alert for Kerala with 100% prediction confidence, prompting the team to activate evacuation protocols.

Scenario 2: An analyst validating the model's specificity in arid regions loads the "Safe Weather Demo" for West Rajasthan. With Jun-Sep cumulative rainfall of only 280 mm, the Decision Tree Classifier correctly returns "No Flood Risk Detected", confirming that the model does not over-trigger warnings in naturally low-rainfall subdivisions.

Scenario 3: A district disaster-management office uses the Use Cases page to plan ahead of the season: predicted flood indices guide Evacuation Planning (identifying shelter zones and escape routes), Resource Allocation (positioning rescue boats, medical staff, and food caches near high-risk nodes), and Model Validation (benchmarking predictions against historical flood records to support policy decisions).

Technical Architecture

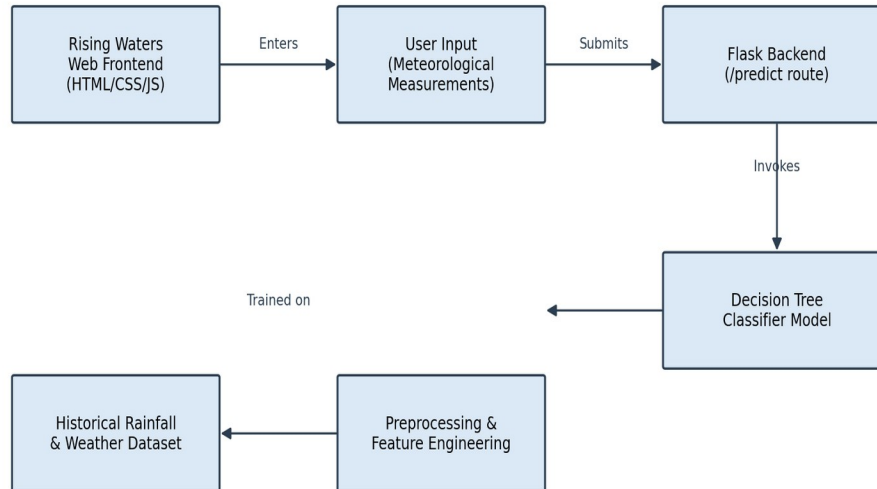


Figure 1: High-level architecture of the Rising Waters platform

Description: Rising Waters follows a modular three-tier architecture. The frontend (HTML, CSS, and JavaScript) provides a responsive, multi-page interface for exploring the project and running predictions. The Flask backend receives user-submitted meteorological measurements at the `/predict` route, validates the inputs, and passes them to the trained Decision Tree Classifier. The model — trained offline on a preprocessed historical rainfall and weather dataset — returns a binary flood-risk classification along with a confidence score, which the backend packages into a summary card rendered back to the user.

Note: The model artifact (e.g. a serialized .pkl file) is loaded once at application startup to keep prediction latency low across repeated requests.

Pre-requisites

- **Flask Framework Knowledge:** Understanding of routing, templates, and request handling in Flask.
- **Machine Learning Fundamentals:** Familiarity with classification algorithms, particularly Decision Trees, and evaluation metrics such as accuracy.
- **Data Handling with pandas & scikit-learn:** Ability to clean, transform, and split tabular meteorological data, and to train/evaluate scikit-learn models.
- **HTML, CSS, and JavaScript Skills:** Needed to build the multi-page responsive frontend, including forms and dynamic result rendering.
- **Python Programming Proficiency:** Core language for both the ML pipeline and the Flask application.
- **Version Control with Git:** For tracking changes across the dataset pipeline, model training scripts, and the web application.
- **Development Environment Setup:** A configured virtual environment with Flask, scikit-learn, pandas, and joblib/pickle installed.

Project Workflow

Milestone 1: Data Collection & Preprocessing

- Activity 1.1: Source historical sub-divisional rainfall and weather records spanning over a century.
- Activity 1.2: Clean the dataset — handle missing values, standardize subdivision names, and align date ranges.
- Activity 1.3: Engineer cumulative rainfall features (annual, Jan–Feb, Mar–May, Jun–Sep, Oct–Dec, average June rainfall, average subdivision rainfall).
- Activity 1.4: Label historical records with a binary flood/no-flood flag for supervised training.

Milestone 2: Model Selection & Training

- Activity 2.1: Evaluate candidate classifiers (Decision Tree, Logistic Regression, Random Forest) on the engineered feature set.
- Activity 2.2: Select the Decision Tree Classifier for its balance of interpretability and accuracy.
- Activity 2.3: Train and validate the model, achieving 96.55% accuracy on held-out historical data.
- Activity 2.4: Serialize the trained model artifact for use by the Flask backend.

Milestone 3: Flask Backend Development

- Activity 3.1: Define the application routes — Home, Prediction, About, Features, Workflow, Use Cases, and Contact.
- Activity 3.2: Implement the /predict POST endpoint to accept form data, validate it, and run the loaded model.
- Activity 3.3: Build response logic returning risk status, confidence percentage, key rainfall metric, and model accuracy.

Milestone 4: Frontend Development

- Activity 4.1: Design a responsive single-page-per-section layout with a persistent navigation bar (Home, Prediction, About, Features, Workflow, Use Cases, Contact).
- Activity 4.2: Build the Flood Risk Assessment form with demo preset buttons ("Load Flood Demo" / "Load Safe Weather Demo") for quick testing.
- Activity 4.3: Create dynamic result banners — a red "Flood Risk Detected" alert and a green "No Flood Risk Detected" confirmation — each with a Prediction Summary panel.

Milestone 5: Deployment & Testing

- Activity 5.1: Run the Flask development server locally and verify all navigation routes and form validations.
- Activity 5.2: Test predictions using both demo presets and manually entered values across multiple subdivisions.
- Activity 5.3: (Optional) Expose the local server publicly via Ngrok for demonstration and stakeholder review.

Milestone 6: Conclusion

Milestone 1: Data Collection & Preprocessing

This milestone establishes the foundation of the entire prediction pipeline: a clean, feature-rich historical dataset covering sub-divisional rainfall and weather patterns across India.

Activity 1.1: Assemble the Historical Dataset

Historical meteorological records are compiled at the sub-divisional level, including average temperature, average humidity, cloud cover, and rainfall totals across multiple time windows. Each historical year/subdivision pair is labeled with whether a flood event occurred, forming the ground truth for supervised learning.

Activity 1.2: Clean and Standardize the Data

- Remove duplicate or malformed rows and correct inconsistent subdivision naming.
- Impute or drop records with missing rainfall or temperature values.
- Convert all rainfall figures to a consistent unit (millimeters).

Activity 1.3: Engineer Cumulative Rainfall Features

The model relies on several derived rainfall windows rather than raw daily data, since seasonal accumulation is far more predictive of flooding than any single day's reading:

- Total Annual Rainfall (mm)
- Jan – Feb Cumulative Rainfall (mm)
- Mar – May Cumulative Rainfall (mm)
- Jun – Sep Cumulative Rainfall (mm) — the strongest predictor, since it captures the core monsoon window
- Oct – Dec Cumulative Rainfall (mm)
- Average June Rainfall (mm)
- Average Subdivision Rainfall (mm) — a long-run baseline used to detect anomalies

Activity 1.4: Split the Dataset

The engineered dataset is split into training and test sets (a common ratio such as 80/20), stratified by the flood/no-flood label to preserve class balance for reliable evaluation.

Milestone 2: Model Selection & Training

Activity 2.1: Evaluate Candidate Models

Several classification approaches are compared on the engineered feature set — Logistic Regression as a simple baseline, Random Forest for ensemble robustness, and a Decision Tree Classifier for its transparent, rule-based decision boundaries that are easy to explain to non-technical stakeholders such as disaster-response teams.

Activity 2.2: Train the Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

model = DecisionTreeClassifier(
    criterion="gini", max_depth=8, random_state=42
)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Model Accuracy: {accuracy * 100:.2f}%")
# Model Accuracy: 96.55%
```

Activity 2.3: Persist the Trained Model

```
import joblib

joblib.dump(model, "flood_model.pkl")
joblib.dump(scaler, "scaler.pkl") # if feature scaling is used
```

The serialized model and any preprocessing objects (such as a scaler or subdivision encoder) are stored alongside the Flask application, ready to be loaded once at server startup rather than re-trained on every request.

Milestone 3: Flask Backend Development

Milestone 3 focuses on wiring the trained model into a Flask application that exposes a clean set of routes for the website's pages and a POST endpoint for running predictions.

Activity 3.1: Define the Core Routes

```
@app.route("/")
def home():
    return render_template("home.html")

@app.route("/predict", methods=["GET", "POST"])
def predict():
    if request.method == "GET":
        return render_template("predict.html")
    # POST: handle form submission (see Activity 3.2)
```

Activity 3.2: Process the Prediction Request

```
@app.route("/predict", methods=["POST"])
def predict():
    form = request.form
    features = [
        float(form["avg_temperature"]),
        float(form["avg_humidity"]),
        float(form["cloud_cover"]),
        float(form["total_annual_rainfall"]),
        float(form["jan_feb_rainfall"]),
        float(form["mar_may_rainfall"]),
        float(form["jun_sep_rainfall"]),
        float(form["oct_dec_rainfall"]),
        float(form["avg_june_rainfall"]),
        float(form["avg_subdivision_rainfall"]),
    ]

    prediction = model.predict(features)[0]
    confidence = model.predict_proba(features).max() * 100

    return render_template(
        "result.html",
        flood_risk=bool(prediction),
        confidence=round(confidence, 2),
        jun_sep_rainfall=form["jun_sep_rainfall"],
        subdivision=form["subdivision"],
        model_accuracy=96.55,
    )
```

Activity 3.3: Validate Inputs

- Ensure all numeric fields are provided and within plausible physical ranges (e.g. humidity 0–100%).
- Return a friendly error message on the Prediction page if required fields are missing or malformed.
- Guard against non-numeric input using try/except around the float() conversions.

Milestone 4: Frontend Development

Milestone 4 focuses on building an informative, easy-to-navigate multi-page interface so that both technical and non-technical users can understand and operate the prediction model.

Activity 4.1: Global Navigation & Layout

- A persistent top navigation bar links to Home, Prediction, About, Features, Workflow, Use Cases, and Contact.
- A blue-and-white color scheme with a water-drop logo reinforces the "Rising Waters" brand across every page.
- A scroll-progress indicator bar runs along the top of the viewport on longer pages.

Activity 4.2: The Prediction Form

- Two preset buttons — "Load Flood Demo" and "Load Safe Weather Demo" — auto-fill the form with realistic sample values for quick testing.
- Input fields cover Meteorological Subdivision, Average Temperature, Average Humidity, Cloud Cover, and six cumulative-rainfall metrics.
- A "Predict Flood Risk" button submits the form, with a "Reset" button to clear all fields.

Activity 4.3: Result Presentation

- A full-width colored banner (red for flood risk, green for no risk) immediately communicates the outcome with a short natural-language explanation naming the subdivision.
- A four-card "Prediction Summary" grid displays the key Jun–Sep cumulative rainfall figure, prediction confidence, the algorithm used (Decision Tree Classifier), and the underlying model's overall accuracy (96.55%).

Activity 4.4: Supporting Content Pages

- About: Explains what Rising Waters is, why flood prediction matters, and how the ML model solves the problem.
- Features: Showcases capabilities such as ML Prediction, Early Warning System, Real-Time Assessment, and Preparedness Support.
- Workflow: Presents the development pipeline as a vertical timeline (Historical Weather Dataset → Data Cleaning → Exploratory Data Analysis → ...).
- Use Cases: Describes real-world applications — Evacuation Planning, Resource Allocation, and Model Validation — followed by an FAQ section.
- Contact: A simple support form collecting Name, Email Address, and Message.

Milestone 5: Deployment & Testing

Activity 5.1: Local Deployment

```
D:\projects> python -m venv myenv
D:\projects> "d:\projects\myenv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
(myenv) D:\projects> python app.py
* Serving Flask app 'app'
* Running on http://127.0.0.1:5000
```

Once running, the application is accessed locally at <http://127.0.0.1:5000>, where all seven pages (Home, Prediction, About, Features, Workflow, Use Cases, Contact) become available for verification.

Activity 5.2: Functional Testing

- Submit the "Load Flood Demo" preset and confirm the red "Flood Risk Detected" banner appears with the correct subdivision name and rainfall figure.
- Submit the "Load Safe Weather Demo" preset and confirm the green "No Flood Risk Detected" banner appears.
- Manually vary the Jun-Sep cumulative rainfall value around the model's internal decision threshold to confirm the classification flips as expected.
- Click through every navigation link (Home, Prediction, About, Features, Workflow, Use Cases, Contact) to confirm each section renders and scrolls correctly.

Activity 5.3: Optional Public Exposure via Ngrok

For demonstrations or stakeholder review, the local Flask server can be tunneled to a public URL using Ngrok, following the same pattern of installing pyngrok, configuring an authtoken, and running a small launch script that starts both the tunnel and the Flask server together.

Exploring the Website's Web Pages

Home Page

The landing page introduces Rising Waters with a clear value proposition, a short description of the ML system, two primary calls to action ("Predict Flood Risk" and "Learn More"), and an "Early Alert System" highlight card.

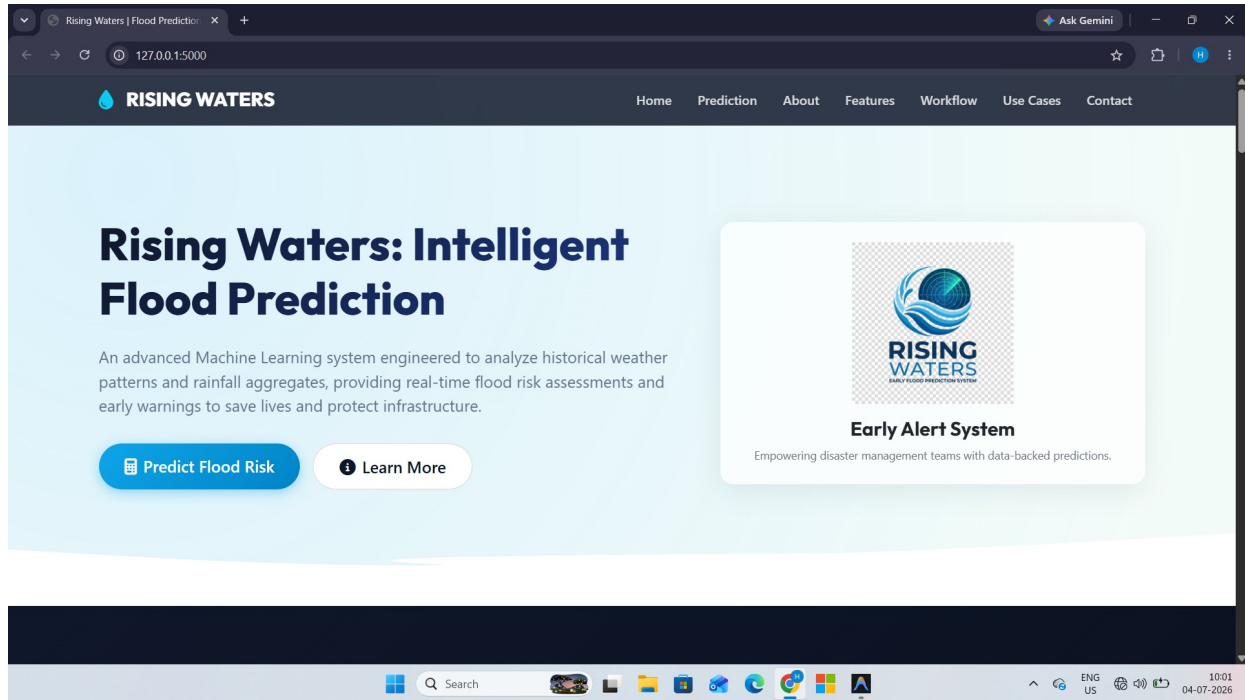


Figure 2: Rising Waters home page

Prediction Page — Input Form

The Flood Risk Assessment form lets users select a Meteorological Subdivision and enter temperature, humidity, cloud cover, and rainfall figures. Two demo buttons auto-fill sample data for quick testing.

The screenshot shows the top section of the 'Flood Risk Assessment' form. At the top, there's a navigation bar with 'RISING WATERS' and links to Home, Prediction, About, Features, Workflow, Use Cases, and Contact. Below this, the title 'Flood Risk Assessment' is centered, followed by a brief instruction: 'Enter the meteorological measurements below to run prediction models. You can also use the preset demo buttons to auto-fill sample measurements.' Two buttons, 'Load Flood Demo' and 'Load Safe Weather Demo', are positioned above the input fields. The form fields are arranged in two columns: 'Meteorological Subdivision' (with a dropdown showing 'Andaman & Nicobar Islands'), 'Average Temperature (°C)' (with a text input showing 'e.g. 28.5'), 'Average Humidity (%)' (with a range input '0 to 100'), 'Cloud Cover (%)' (with a range input '0 to 100'), 'Total Annual Rainfall (mm)' (with a text input showing 'e.g. 3200'), and 'Jan - Feb Cumulative Rainfall (mm)' (with a text input showing 'e.g. 25').

Figure 3: Prediction form — top section with demo presets

The screenshot shows the bottom section of the 'Flood Risk Assessment' form. It continues with 'Mar - May Cumulative Rainfall (mm)' (text input 'e.g. 150'), 'Jun - Sep Cumulative Rainfall (mm)' (text input 'Threshold: > 1091.89mm triggers warning'), 'Oct - Dec Cumulative Rainfall (mm)' (text input 'e.g. 300'), and 'Average June Rainfall (mm)' (text input 'e.g. 400'). Below these is 'Average Subdivision Rainfall (mm)' (text input 'e.g. 2600'). At the bottom, there are two buttons: a large blue 'Predict Flood Risk' button and a smaller 'Reset' button. A blue circular arrow icon is visible in the bottom right corner of the form area.

Figure 4: Prediction form — remaining rainfall fields and submit controls

Prediction Results

After submission, the model's classification is presented immediately as a full-width banner, followed by a four-card Prediction Summary.

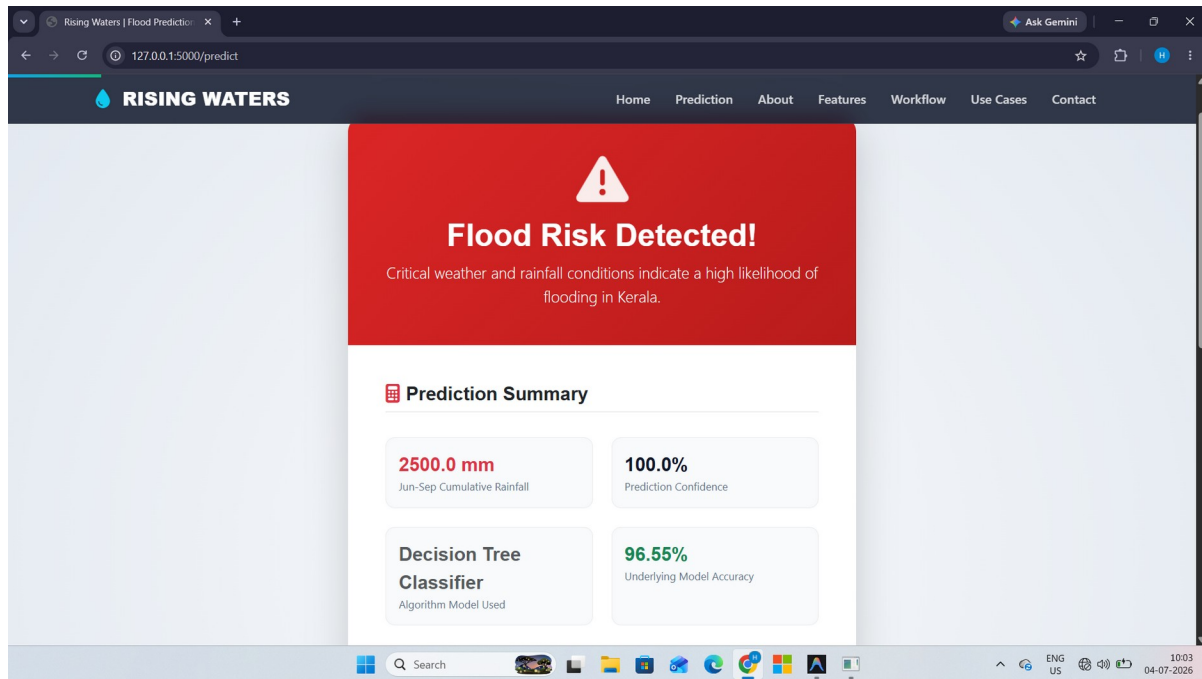


Figure 5: "Flood Risk Detected" result for Kerala

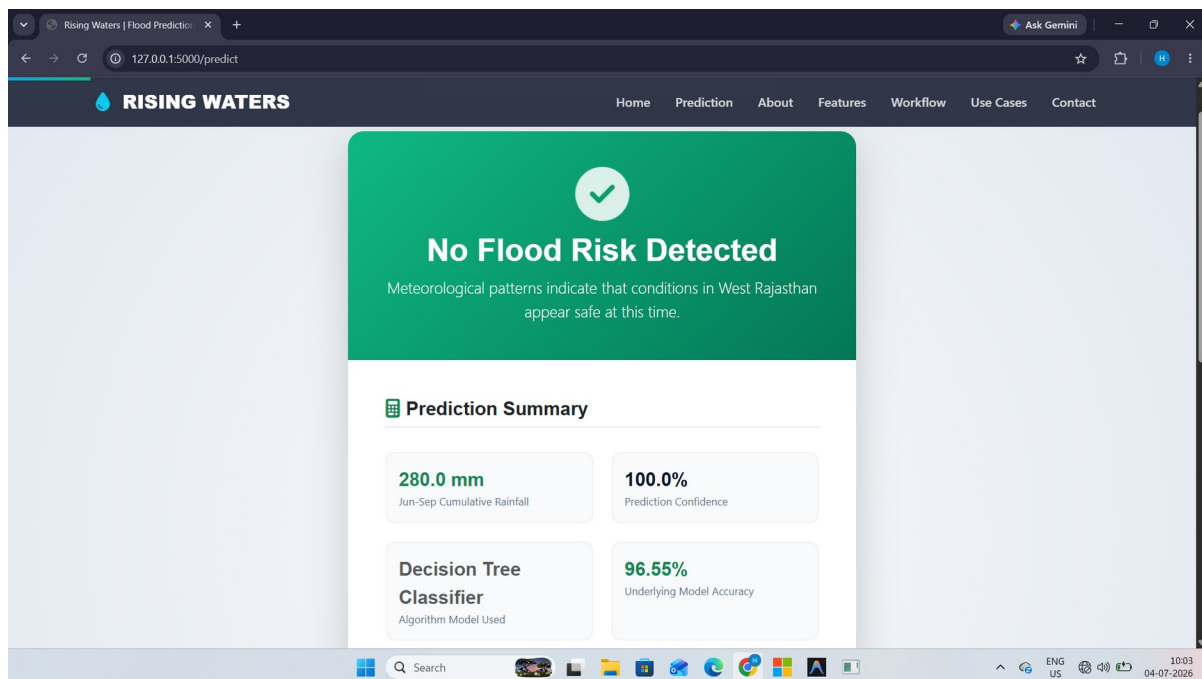


Figure 6: "No Flood Risk Detected" result for West Rajasthan

About Page

The About section explains what Rising Waters is, why flood prediction matters for emergency responders, and how the underlying Decision Tree Classifier examines sub-divisional rainfall distributions to recognize high-risk trends.

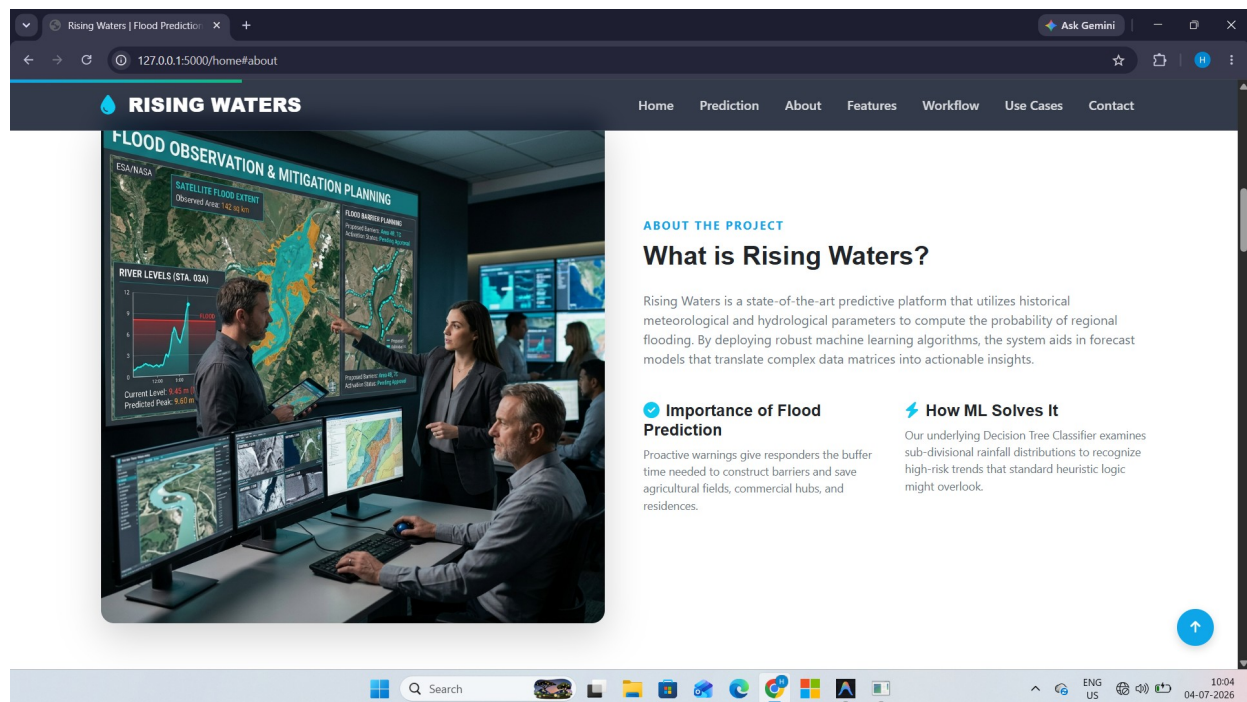


Figure 7: About page — "What is Rising Waters?"

Features Page

The Key System Features section showcases the platform's core capabilities, including ML Prediction, an Early Warning System, Real-Time Assessment, and Preparedness Support, each presented as an icon-led card.

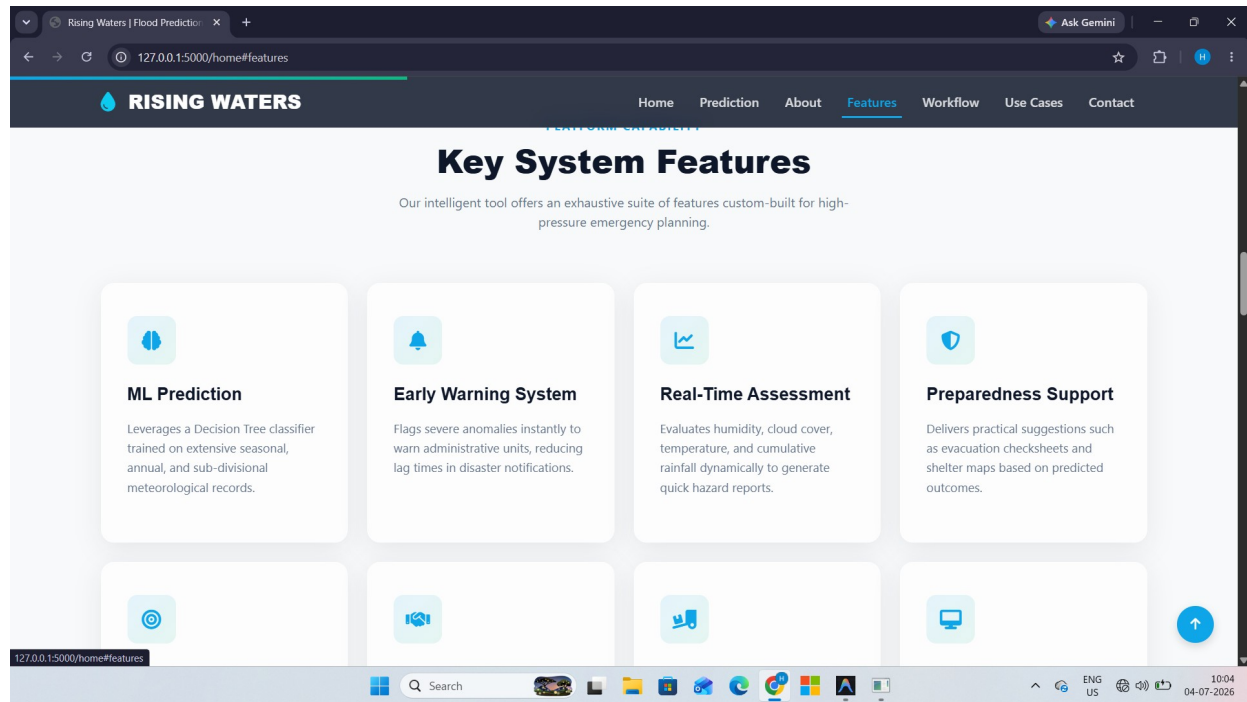


Figure 8: Features page — Key System Features

Workflow Page

The Development Workflow page presents the machine learning pipeline as a vertical timeline, starting from the Historical Weather Dataset, through Data Cleaning and Exploratory Data Analysis, to the final deployed model.

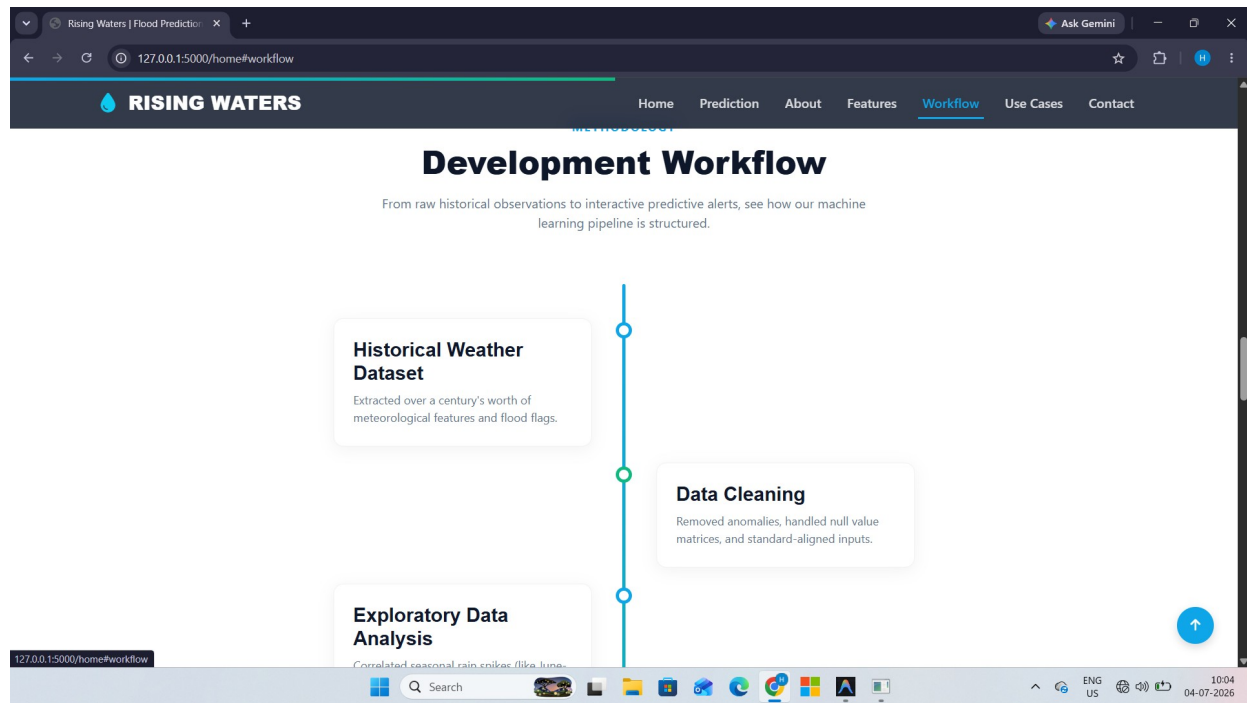


Figure 9: Workflow page — Development Workflow timeline

Use Cases Page

Real-World Scenarios highlights how stakeholders deploy Rising Waters for Evacuation Planning, Resource Allocation, and Model Validation, followed by a Frequently Asked Questions section covering system performance and input methods.

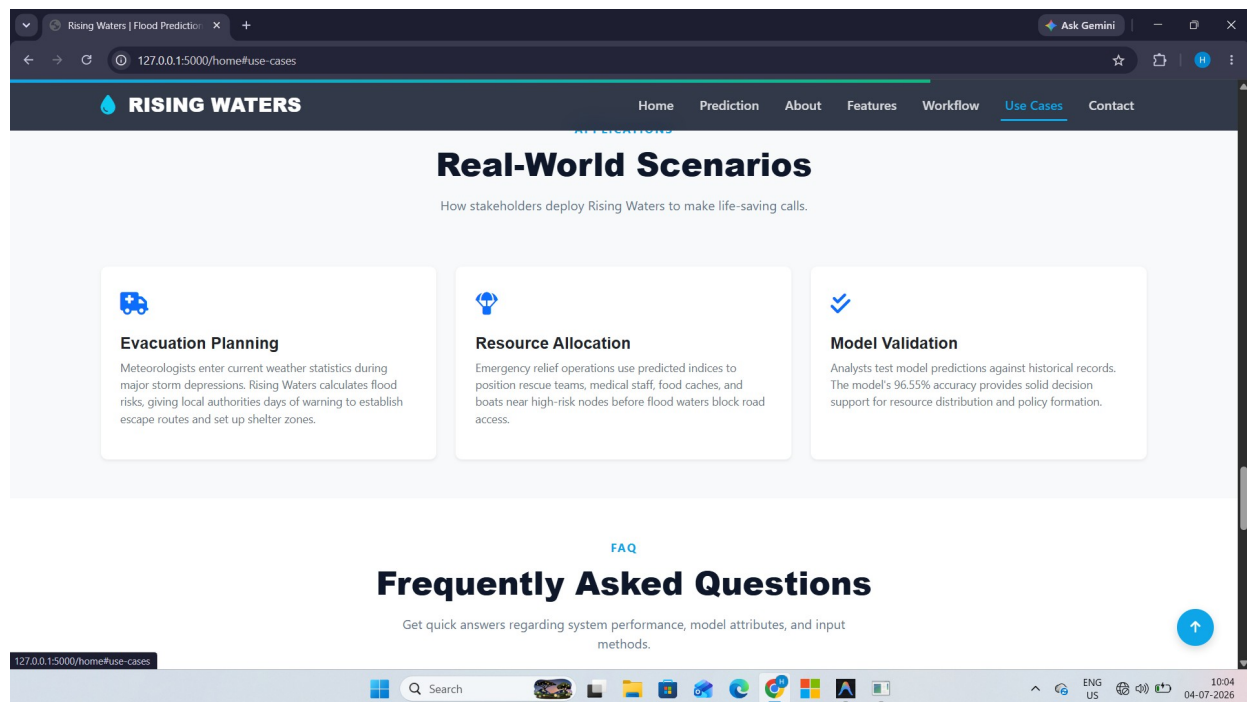
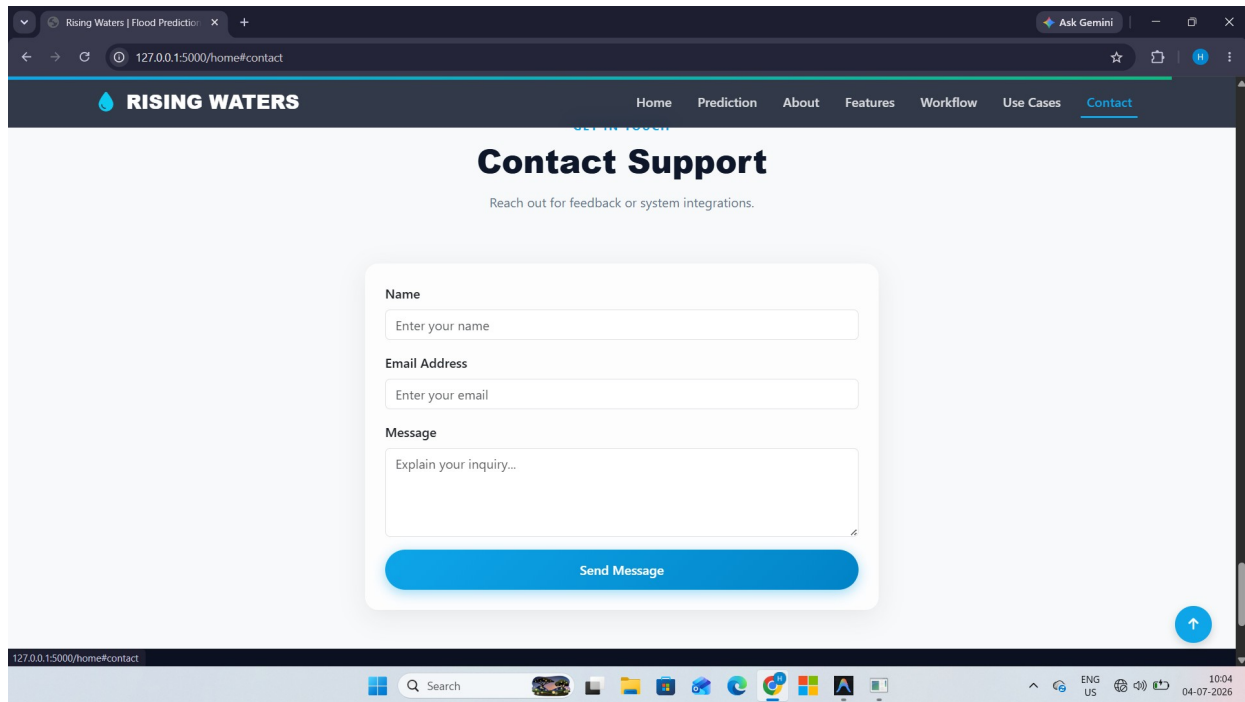


Figure 10: Use Cases page — Real-World Scenarios and FAQ

Contact Page

The Contact Support page offers a simple feedback and integrations form collecting Name, Email Address, and Message, giving users a direct channel to reach the Rising Waters team.



The screenshot shows a web browser window displaying the 'Contact Support' page of the 'Rising Waters' application. The browser's address bar shows the URL '127.0.0.1:5000/home#contact'. The page has a dark blue header with the 'RISING WATERS' logo and a navigation menu with links to Home, Prediction, About, Features, Workflow, Use Cases, and Contact. The main content area is light gray and features a white form titled 'Contact Support' with the subtitle 'Reach out for feedback or system integrations.' The form contains three input fields: 'Name' (placeholder: 'Enter your name'), 'Email Address' (placeholder: 'Enter your email'), and 'Message' (placeholder: 'Explain your inquiry...'). A blue 'Send Message' button is positioned below the message field. A blue circular button with an upward arrow is located in the bottom right corner of the form area. The browser's taskbar at the bottom shows the Windows logo, a search bar, and various application icons, with the system clock indicating 10:04 on 04-07-2026.

RISING WATERS

Home Prediction About Features Workflow Use Cases **Contact**

Contact Support

Reach out for feedback or system integrations.

Name

Email Address

Message

Send Message

Figure 11: Contact Support page

Conclusion

Rising Waters is a Flask-based flood prediction platform that pairs a Decision Tree Classifier — trained on over a century of historical sub-divisional rainfall and weather records — with a clear, multi-page web experience. Achieving 96.55% accuracy, the model translates raw meteorological inputs into an instant, color-coded flood-risk verdict, a confidence score, and the key rainfall figure driving the decision, giving disaster-management teams the lead time needed for evacuation planning and resource allocation.

The project's structured workflow — from historical data collection and feature engineering, through model training, Flask backend development, frontend design, and deployment — demonstrates how a focused ML pipeline can be packaged into an accessible tool for meteorologists, emergency responders, and policymakers.

Looking ahead, Rising Waters could be extended with real-time weather API integration for automatic input population, satellite-based flood-extent overlays on an interactive map, SMS/email early-warning alerts to registered administrators, and a historical dashboard tracking prediction accuracy over successive monsoon seasons. These enhancements would help the platform evolve from a single-prediction tool into a continuously monitoring, region-wide flood early-warning system.